

SEMANA 1: CIBERSEGURIDAD.

1.1. Modo Razonamiento: La Lógica del Plano de Red

En BlackArch, no tenemos interfaces gráficas que nos oculten la realidad. Aquí, los modelos OSI y TCP/IP son nuestra "visión de rayos X".

- **El Atacante (The Red Teamer):** Ve cada capa como una vulnerabilidad potencial. Si el objetivo tiene un firewall de Capa 4 (TCP/UDP) muy estricto, el atacante intentará **encapsular** su tráfico malicioso dentro de protocolos de Capa 7 (como DNS o HTTP) para que el firewall lo deje pasar pensando que es tráfico web normal.
- **El Defensor (The Blue Teamer):** Usa el modelo OSI para aislar problemas. Si un servidor no responde, el defensor verifica: ¿Hay enlace físico? (Capa 1), ¿Tiene IP y responde a ping? (Capa 3), ¿El puerto está abierto? (Capa 4), ¿El servicio devuelve un error de aplicación? (Capa 7).

Imagina que la red es un edificio de alta seguridad:

- **Capa 2 (Enlace):** Es la recepción. Si el atacante engaña al recepcionista (Switch), puede hacerse pasar por el gerente (Router) mediante ARP Spoofing.
- **Capa 3 (Red):** Son los pasillos. Aquí el atacante puede usar IP Spoofing para enviar paquetes que parecen venir de una zona de confianza.
- **Capa 4 (Transporte):** Son las puertas de las oficinas (Puertos). El atacante las "golpea" todas (Port Scanning) para ver cuál se quedó abierta.
- **Capa 7 (Aplicación):** Es lo que sucede dentro de la oficina. Aquí es donde se roban las contraseñas o se inyecta código malicioso.

Tu misión como defensor: Bloquear cada capa para que, si una falla, la otra resista.

Práctica: Auditoría de la Pila de Red en BlackArch

En BlackArch, utilizaremos herramientas de línea de comandos para inspeccionar cómo interactúan estas capas en tu máquina de pruebas.

1. Identificación de la Capa 2 (Direcciones Físicas)

Antes de atacar o defender, debemos saber quiénes somos en la red local. La dirección MAC es nuestra identidad "física" que no debería cambiar.

Listar interfaces y ver la dirección MAC (link/ether)

ip link show

```
[ jo4nlu-machine ~ ]$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT group default qlen 1000
   link/ether bc:f4:d4:9f:7b:7d brd ff:ff:ff:ff:ff:ff
3: enp0s20f0u4u5c2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000
   link/ether c8:17:f5:3e:f3:38 brd ff:ff:ff:ff:ff:ff
   altname enx817f53ef338
```

2. Capa 2 (Enlace) y Capa 3 (Red)

Para que un ataque como el **ARP Spoofing** funcione, el atacante debe manipular la tabla de vecinos. Vamos a ver cómo tu sistema mapea direcciones físicas (MAC) con lógicas (IP).

- # Ver interfaces de red y sus direcciones MAC/IP (Capa 2 y 3)
- # En BlackArch verás interfaces como eth0 o wlan0

ip addr show

```
[ jo4nlu-machine ~ ]$ # Ver interfaces de red y sus direcciones MAC/IP (Capa 2 y 3)
# En BlackArch verás interfaces como eth0 o wlan0
ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:f4:d4:9f:7b:7d brd ff:ff:ff:ff:ff:ff
    inet 172.20.10.5/28 brd 172.20.10.15 scope global dynamic noprefixroute wlan0
        valid_lft 2814sec preferred_lft 2814sec
    inet6 fe80::6602:39f5:244e:c282/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s20f0u4u5c2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether c8:17:f5:3e:f3:38 brd ff:ff:ff:ff:ff:ff
    altname enx817f53ef338
```

- # Ver la tabla ARP/Neighbor (Mapeo Capa 3 -> Capa 2)
- # Esta tabla es el objetivo principal de ataques de interceptación local

ip neigh show

```
[ jo4nlu-machine ~ ]$ ip neigh show
172.20.10.1 dev wlan0 lladdr b6:40:a4:67:92:64 REACHABLE
fe80::b440:a4ff:fe67:9264 dev wlan0 lladdr b6:40:a4:67:92:64 router STALE
[ jo4nlu-machine ~ ]$
```

3. Capa 4 (Transporte)

Aquí es donde controlamos los **Sockets**. Un socket es la unión de una IP y un Puerto. Como defensores, nuestra regla de oro es: *"Si el puerto no se usa, debe estar cerrado"*.

- # Listar todos los puertos TCP (-t) y UDP (-u) abiertos y en escucha (-l)
- # Mostrando el proceso (-p) y de forma numérica (-n)
- # Útil para detectar Backdoors o servicios innecesarios

sudo ss -tunlp

[jo4nlu-machine ~]\$ sudo ss -tunlp				Aquí es donde controlamos los Sockets. Un socket es la unión de una IP y un Puerto. Como defensores, nuestra regla de oro es: "Si un puerto está escuchando, debe estar cerrado".			
[sudo] contraseña para jo4nlu:				Local Address:Port		Peer Address:Port	
Netid	State	Recv-Q	Send-Q				
Process							
udp	UNCONN	0	0	127.0.1.1:13012	0.0.0.0:*		
users:(("DesktopEditors", pid=6666, fd=9))							
udp	UNCONN	0	0	224.0.0.251:5353	0.0.0.0:*		
users:(("brave", pid=4910, fd=71))							
udp	UNCONN	0	0	224.0.0.251:5353	0.0.0.0:*		
users:(("brave", pid=4910, fd=50))							
udp	UNCONN	0	0	224.0.0.251:5353	0.0.0.0:*		
users:(("brave", pid=4841, fd=127))							
udp	UNCONN	0	0	127.0.0.1:5353	0.0.0.0:*		
users:(("tor", pid=982, fd=7))							
udp	UNCONN	0	0	[fe80::6602:39f5:244e:c282]%%wlan0:546	[::]:*		
users:(("NetworkManager", pid=784, fd=30))							
tcp	LISTEN	0	80	0.0.0.0:3306	0.0.0.0:*		
users:(("mariadb", pid=1004, fd=32))							
tcp	LISTEN	0	32768	127.0.0.1:9040	0.0.0.0:*		
users:(("tor", pid=982, fd=8))							
tcp	LISTEN	0	32768	127.0.0.1:9050	0.0.0.0:*		
users:(("tor", pid=982, fd=6))							
tcp	LISTEN	0	80	[::]:3306	[::]:*		
users:(("mariadb", pid=1004, fd=33))							
tcp	LISTEN	0	511	*:80	*:*		
users:(("httpd", pid=1007, fd=4), ("httpd", pid=1006, fd=4), ("httpd", pid=1005, fd=4), ("httpd", pid=954, fd=4))							

3. Análisis de Capas en Tiempo Real (Sniffing)

Para entender el encapsulamiento, usaremos tcpdump. Veremos cómo un simple paquete tiene información de múltiples capas.

Bash

- # Capturar solo 1 paquete para analizar su estructura
- # -X muestra el contenido en Hex/ASCII (Capa 7)
- # -vv da máxima verbosidad (detalles de Capas 3 y 4)

sudo tcpdump -i any -c 1 -X

[jo4nlu-machine ~]\$ sudo tcpdump -i any -c 1 -X				Bash			
[sudo] contraseña para jo4nlu:				tcpdump: WARNING: any: That device doesn't support promiscuous mode			
(Promiscuous mode not supported on the "any" device)				Capturar solo 1 paquete para analizar su estructura			
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode				Hex/ASCII (Capa 7)			
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes				Cajón de sastre			
10:26:13.695890 wlan0 Out IP jo4nlu-machine.38332 > cj-in-f95.1e100.net.https: UDP, length 2				Cajas 3 y 4			
0x0000: 4500 0039 c023 4000 4011 351d ac14 0a05 E..9.#@.@.5.....				Notas			
0x0010: 8efb 005f 95bc 01bb 0025 6839 48f2 beb4 ..._.....%h9H...				2.2.2.2			
0x0020: 5d4f a810 2173 3e2e f210 0ccf 12a9 4894]0..!s>.....H.				Notas			
0x0030: 6be4 3261 c7b4 58bf 35				Notas			
1 packet captured				Encapsulamiento			
4 packets received by filter				Es el proceso donde los datos viajan por las			
0 packets dropped by kernel				cabeceras de la Capa 4. Luego en la 3 y finalmente en la 2.			
[jo4nlu-machine ~]\$				de palabras			

Modo Razonamiento: El "Puente" entre Capas

Aunque el **Router** es un dispositivo de **Capa 3 (Red)** y usa **IPs**, el ataque de ver dos IPs con la misma MAC ocurre en la **Capa 2 (Enlace de Datos)**.

¿Por qué? El protocolo **ARP (Address Resolution Protocol)** es el "pegamento". Su único trabajo es preguntar: "Oye, ¿quién tiene la IP X? Dime tu dirección MAC".

- **La Lógica del Atacante:** El atacante no ataca la IP (Capa 3); lo que hace es **mentirle** a tu tarjeta de red sobre la identidad física (Capa 2). Envía paquetes ARP falsos diciendo: "Yo soy el Router (IP 192.168.1.1), mi MAC es la de mi laptop de hacker".
- **El resultado en tu Word:** Tu tabla ARP se confunde. Para tu PC, el "camino físico" hacia Internet ahora pasa por la dirección MAC del atacante.

Conclusión para tu reporte: El problema se manifiesta en la **Capa 2**, porque es una vulnerabilidad de confianza en las direcciones MAC. El Switch (que es de Capa 2) simplemente reenvía los datos a donde la MAC le diga, sin verificar si esa IP realmente pertenece a ese equipo.

Práctica en BlackArch: Análisis de la Mentira (ARP)

Para que tu reporte sea 100% práctico, vamos a usar una herramienta específica de BlackArch para ver cómo se ve esta resolución de nombres.

1. Ver el protocolo ARP en acción

En lugar de solo ver la tabla, vamos a ver las "preguntas y respuestas" de la red.

Instalamos/verificamos una herramienta básica de diagnóstico si no la tienes
(BlackArch suele tener 'iproute2' por defecto)

Este comando limpia tu caché ARP para forzar al sistema a preguntar de nuevo
(CUIDADO: Esto puede desconectarte un segundo si estás por SSH)

sudo ip -s -s neigh flush all

Ahora, mira cómo el sistema vuelve a aprender quién es quién

ip neigh show

```
[ jo4nlu-machine ~ ]$ # Instalamos/verificamos una herramienta básica de diagnóstico si no la tienes
# (BlackArch suele tener 'iproute2' por defecto)

# Este comando limpia tu caché ARP para forzar al sistema a preguntar de nuevo
# (CUIDADO: Esto puede desconectarte un segundo si estás por SSH)
sudo ip -s -s neigh flush all

# Ahora, mira cómo el sistema vuelve a aprender quién es quién
ip neigh show
[sudo] contraseña para jo4nlu:
172.20.10.1 dev wlan0 lladdr b6:40:a4:67:92:64 ref 1 used 515/2/515probes 1 REACHABLE
fe80::b440:a4ff:fe67:9264 dev wlan0 lladdr b6:40:a4:67:92:64 router used 6885/6885/6869probes 1 STALE

*** Round 1, deleting 2 entries ***
*** Flush is complete after 1 round ***
[ jo4nlu-machine ~ ]$
```

Claro, directo al grano:

- Eliminaste la caché ARP y el sistema la reconstruyó bien.
- 172.20.10.1 → tu router, estado REACHABLE (todo OK).
- La dirección IPv6 → mismo router, estado STALE (normal, no es error).

- Se borraron 2 entradas y se regeneraron sin problema.

Conclusión: tu red funciona нормально, sin fallos ni conflictos.

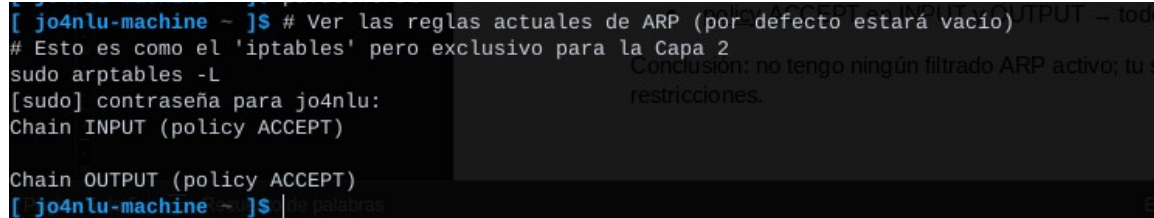
2. Inspección profunda con arptables (El Firewall de Capa 2)

BlackArch te permite incluso filtrar quién puede hablarte en Capa 2. Esto es puro *Hardening*.

Ver las reglas actuales de ARP (por defecto estará vacío)

Esto es como el 'iptables' pero exclusivo para la Capa 2

sudo arptables -L



```
[ jo4nlu-machine ~ ]$ # Ver las reglas actuales de ARP (por defecto estará vacío)
# Esto es como el 'iptables' pero exclusivo para la Capa 2
sudo arptables -L
[sudo] contraseña para jo4nlu:
Chain INPUT (policy ACCEPT)
Chain OUTPUT (policy ACCEPT)
[ jo4nlu-machine ~ ]$
```

Conclusión: no tengo ningún filtrado ARP activo; tu sistema acepta y envía ARP sin restricciones.

Interpretación rápida:

- arptables está vacío → no hay reglas aplicadas.
- policy ACCEPT en INPUT y OUTPUT → todo el tráfico ARP está permitido.

Conclusión: no tengo ningún filtrado ARP activo; tu sistema acepta y envía ARP sin restricciones.

Modo Razonamiento: La Máscara y el "Portero" (Gateway)

¿Recuerdas que te pregunté cómo sabía tu PC si una IP estaba en tu red o fuera? Aquí está la lógica que debes anotar:

1. **La Operación AND:** Tu computadora toma su propia IP y la compara con la máscara de subred. Luego toma la IP destino y hace lo mismo.
 - o Si el resultado es igual: **"Está en mi casa"** -> Envía un paquete ARP para buscar la MAC del vecino (Capa 2).
 - o Si el resultado es distinto: **"Está fuera"** -> Envía el paquete a la MAC del **Gateway** (Capa 3 -> Capa 2).
2. **El "Secuestro" del Gateway:** Un atacante no necesita cambiar tu IP. Solo necesita convencerte de que **su MAC** es la del Gateway. Así, aunque tú creas que vas hacia internet, todo pasa por su máquina.

Práctica en BlackArch: Auditoría de Rutas y Conectividad

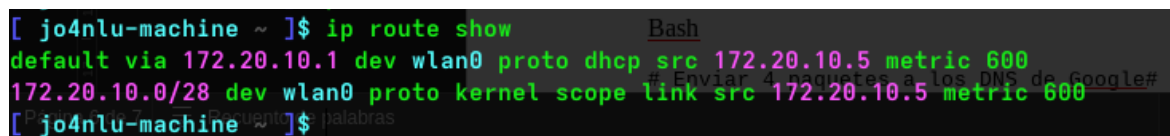
Para tu reporte, vamos a verificar quién es tu "puerta de salida" y cómo se ve ese camino.

1. Ver la Tabla de Rutas (Capa 3)

Este comando te dice a dónde van los paquetes cuando el destino no es local:

Ver la tabla de rutas IP

ip route show



```
[ jo4nlu-machine ~ ]$ ip route show
default via 172.20.10.1 dev wlan0 proto dhcp src 172.20.10.5 metric 600
172.20.10.0/28 dev wlan0 proto kernel scope link src 172.20.10.5 metric 600
```

- default via 172.20.10.1 → tu router (salida a internet)
- dev wlan0 → estás usando WiFi
- src 172.20.10.5 → tu IP
- 172.20.10.0/28 → tu red local (pequeña)

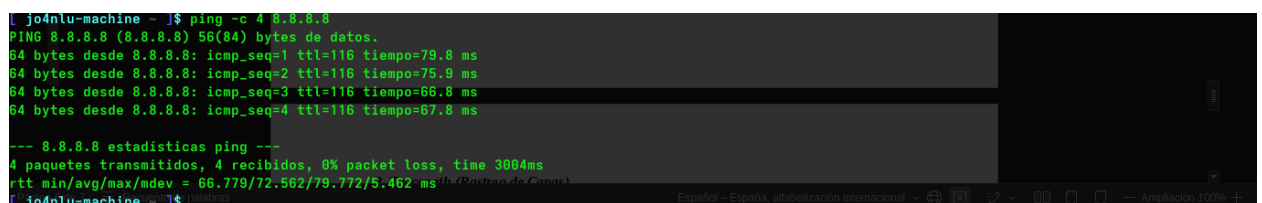
Conclusión: conexión correcta, todo el tráfico pasa por tu router.

2. Diagnóstico de Capa 3 (ICMP)

El comando ping usa el protocolo ICMP. Como mentor, te digo: **No es solo para ver si hay internet**. Es para medir latencia y pérdida de paquetes (indicadores de un ataque DoS o una interceptación lenta).

Enviar 4 paquetes a los DNS de Google# Analiza el TTL (Time To Live): nos dice cuántos "saltos" (routers) cruza

ping -c 4 8.8.8.8



```
[ jo4nlu-machine ~ ]$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes de datos:
64 bytes desde 8.8.8.8: icmp_seq=1 ttl=116 tiempo=79.8 ms
64 bytes desde 8.8.8.8: icmp_seq=2 ttl=116 tiempo=75.9 ms
64 bytes desde 8.8.8.8: icmp_seq=3 ttl=116 tiempo=66.8 ms
64 bytes desde 8.8.8.8: icmp_seq=4 ttl=116 tiempo=67.8 ms

--- 8.8.8.8 estadísticas ping ---
4 paquetes transmitidos, 4 recibidos, 0% packet loss, time 3804ms
rtt min/avg/max/mdev = 66.779/72.562/79.772/5.462 ms
[ jo4nlu-machine ~ ]$
```


Interpretación corta:

- ✓ Respuestas de 8.8.8.8 (Google) → tienes internet
- ✓ 0% packet loss → conexión estable
- ⌚ ~66–80 ms → latencia normal (un poco alta pero ok)

Conclusión: internet funcionando correctamente sin pérdidas.

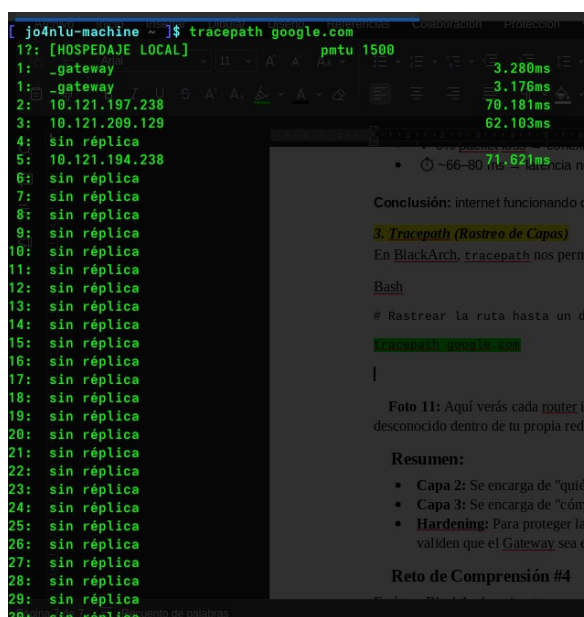
3. Tracpath (Rastreo de Capas)

En BlackArch, tracpath nos permite ver cada "parada" de Capa 3 que hace nuestro paquete.

Bash

Rastrear la ruta hasta un dominio externo

`tracpath google.com`



```
[jo4nlu-machine ~]$ tracpath google.com
1?: [HOSPEDAJE LOCAL] pmtu 1500
1: _gateway 3.280ms
2: 10.121.197.238 3.176ms
3: 10.121.209.129 70.181ms
4: sin réplica 62.103ms
5: 10.121.194.238 71.621ms
6: sin réplica
7: sin réplica
8: sin réplica
9: sin réplica
10: sin réplica
11: sin réplica
12: sin réplica
13: sin réplica
14: sin réplica
15: sin réplica
16: sin réplica
17: sin réplica
18: sin réplica
19: sin réplica
20: sin réplica
21: sin réplica
22: sin réplica
23: sin réplica
24: sin réplica
25: sin réplica
26: sin réplica
27: sin réplica
28: sin réplica
29: sin réplica
30: sin réplica
```

Conclusión: internet funcionando c

3. Tracpath (Rastreo de Capas)

En BlackArch, tracpath nos perm

Bash

Rastrear la ruta hasta un d

`tracpath google.com`

Foto 11: Aquí verás cada router in
desconocido dentro de tu propia red.

Resumen:

- Capa 2: Se encarga de "quién es quién" en el mismo salón (MAC).
- Capa 3: Se encarga de "cómo llego a otro edificio" (IP/Rutas).
- Hardening: Para proteger la Capa 3, configuramos rutas estáticas o usamos firewalls que validen que el Gateway sea el legítimo.

Reto de Comprensión #4

 **Foto 11:** Aquí verás cada router intermedio. En seguridad, si ves un "salto" extra o desconocido dentro de tu propia red, es señal de un dispositivo no autorizado (Rogue Device).



Resumen:

- **Capa 2:** Se encarga de "quién es quién" en el mismo salón (MAC).
- **Capa 3:** Se encarga de "cómo llego a otro edificio" (IP/Rutas).
- **Hardening:** Para proteger la Capa 3, configuramos rutas estáticas o usamos firewalls que validen que el Gateway sea el legítimo.



Reto de Comprensión #4

Estás en BlackArch y ejecutas `tracpath 8.8.8.8`. El **primer salto** que aparece no es la IP de tu router (172.20.10.1), sino una IP desconocida (172.20.10.5).

1. ¿Qué crees que está pasando a nivel de red?
2. ¿En qué capa del modelo OSI deberías empezar a investigar para encontrar al culpable?

Pilar 01: Network Security Hardening

Clase 1.2: Análisis de protocolos: ARP (Ataques y Defensas)

Modo Razonamiento: El talón de Aquiles de la Red Local

El protocolo **ARP** es "confianzudo" por diseño. No tiene un mecanismo para verificar si quien responde es quien dice ser.

- **La Lógica del Atacante:** "Si nadie pregunta, yo respondo igual". El atacante envía paquetes *ARP Reply* sin que nadie se los pida (*Gratuitous ARP*), diciendo: "Yo soy el Router". Tu PC, que es educada, actualiza su tabla y le envía todo el tráfico a él.
- **La Lógica del Defensor:** "Confía, pero verifica". Vamos a aprender a detectar estas mentiras y a ponerle "llave" a nuestra tabla ARP para que nadie la manipule.

Práctica en BlackArch: Detectando e Inmovilizando ARP

Para tu reporte en Word, vamos a usar herramientas de BlackArch para ver la vulnerabilidad y luego aplicar el Hardening.

1. Arping: El escáner de identidades

A diferencia del ping normal (Capa 3), arping trabaja en la **Capa 2**. Sirve para ver si hay dos dispositivos reclamando la misma IP.

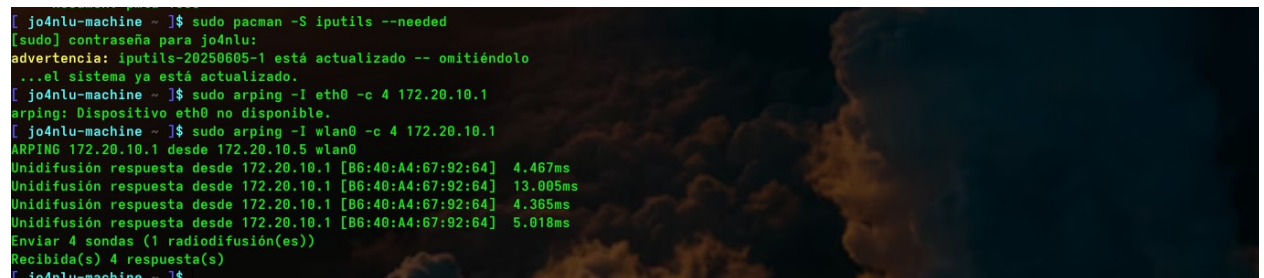
Bash

Instalamos la herramienta si no está (común en el set de iputils)

```
sudo pacman -S iputils --needed
```

Preguntar por la MAC del router (172.20.10.1) # -I especifica la interfaz, -c el número de paquetes

```
sudo arping -I wlan0 -c 4 172.20.10.1
```



```
[ jo4nlu-machine ~ ]$ sudo pacman -S iputils --needed
[sudo] contraseña para jo4nlu:
advertencia: iputils-20250605-1 está actualizado -- omitiéndolo
...el sistema ya está actualizado.
[ jo4nlu-machine ~ ]$ sudo arping -I eth0 -c 4 172.20.10.1
arping: Dispositivo eth0 no disponible.
[ jo4nlu-machine ~ ]$ sudo arping -I wlan0 -c 4 172.20.10.1
ARPING 172.20.10.1 desde 172.20.10.5 wlan0
Unidifusión respuesta desde 172.20.10.1 [B6:40:A4:67:92:64] 4.467ms
Unidifusión respuesta desde 172.20.10.1 [B6:40:A4:67:92:64] 13.005ms
Unidifusión respuesta desde 172.20.10.1 [B6:40:A4:67:92:64] 4.365ms
Unidifusión respuesta desde 172.20.10.1 [B6:40:A4:67:92:64] 5.018ms
Enviar 4 sondas (1 radiodifusión(es))
Recibida(s) 4 respuesta(s)
[ jo4nlu-machine ~ ]$
```

Interpretación corta:

- Respuestas de 172.20.10.1 → tu router responde correctamente
- MAC B6:40:A4:67:92:64 → identificado en la red
- Tiempos 4–13 ms → comunicación rápida (normal en LAN)
- 4 enviadas / 4 recibidas → sin pérdida

Conclusión:

Tu conexión local (WiFi ↔ router) está funcionando perfectamente.

2. Hardening: Tablas ARP Estáticas

Si tienes un servidor crítico en BlackArch y no quieres que nadie desvíe su tráfico, la solución más radical y efectiva es "congelar" la relación IP-MAC.

Bash

1. Miramos la MAC actual del router

```
ip neigh show | grep 172.20.10.1
```

```
[ jo4nlu-machine ~ ]$ ip neigh show | grep 172.20.10.1
172.20.10.1 dev wlan0 lladdr b6:40:a4:67:92:64 REACHABLE
[ jo4nlu-machine ~ ]$
```

Interpretación corta:

- 172.20.10.1 → tu router
- lladdr b6:40:a4:67:92:64 → su dirección MAC
- dev wlan0 → conectado por WiFi
- REACHABLE → está activo y responde

Conclusión:

Tu sistema conoce correctamente al router y la conexión está normal.

2. Convertimos esa entrada en ESTÁTICA (permanent)

Reemplaza [MAC_DEL_ROUTER] por la que obtuviste arriba

```
sudo ip neighbor add 172.20.10.1 lladdr b6:40:a4:67:92:64 dev wlan0 nud permanent
```

```
[ jo4nlu-machine ~ ]$ sudo ip neighbor add 172.20.10.1 lladdr b6:40:a4:67:92:64 dev wlan0 nud permanent
RTNETLINK answers: File exists
[ jo4nlu-machine ~ ]$
```

Significa que ya existe una entrada ARP para esa IP (172.20.10.1) en tu sistema, probablemente con estado:

- REACHABLE o STALE
- o incluso ya estaba como estática o semiestática

3. Verificamos que ahora diga 'PERMANENT'

```
ip neigh show
```

```
[ jo4nlu-machine ~ ]$ ip neigh show
172.20.10.1 dev wlan0 lladdr b6:40:a4:67:92:64 REACHABLE
fe80::b440:a4ff:fe67:9264 dev wlan0 lladdr b6:40:a4:67:92:64 router STALE
[ jo4nlu-machine ~ ]$
```

Conclusión para clase

- La resolución ARP IPv4 funciona correctamente
- El router responde sin pérdida
- No existe conflicto de IP o MAC
- La red local está estable
- IPv6 está presente pero en estado pasivo (STALE)

Resumen:

La tabla ARP del sistema muestra una asociación correcta entre la IP del gateway 172.20.10.1 y su dirección MAC b6:40:a4:67:92:64 en estado REACHABLE, lo que confirma conectividad estable en capa 2. La entrada IPv6 link-local del router se encuentra en estado STALE, lo cual es normal dentro del mecanismo de caché del sistema operativo.

3. Monitoreo Pasivo con arpwat

Un buen defensor no solo bloquea, también vigila. arpwat genera logs cada vez que una IP cambia de MAC en la red.

Bash

```
# Instalar arpwat
```

```
sudo pacman -S arpwat
```

```
# Ejecutarlo en tu interfaz para empezar a vigilar
```

```
# Los cambios se registrarán en /var/log/messages o journalctl
```

```
sudo arpwat -i wlan0
```

```
[ jo4nlu-machine ~ ]$ sudo arpwat -i wlan0 -d
arpwat: Warning: Not compiled with -DDEBUG
[ jo4nlu-machine ~ ]$
```

Interpretación:

- arpwat se inicia en modo **monitoreo pasivo**
- La interfaz wlan0 es la que está siendo vigilada
- El sistema empieza a observar la relación **IP ↔ MAC** en la red local

```
# Ver si hay alertas (esto en una red real es oro puro)
```

```
journalctl -u arpwat -f
```

```
[ jo4nlu-machine ~ ]$ journalctl -u arpwat -f
```

```
[ jo4nlu-machine ~ ]$ journalctl -u arpwat -f
^C[ jo4nlu-machine ~ ]$ journalctl -f
abr 25 18:26:45 jo4nlu-machine sudo[640571]: pam_unix(sudo:session): session opened for user root(uid=0) by jo4nlu(uid=1001)
abr 25 18:26:45 jo4nlu-machine sudo[640571]: pam_unix(sudo:session): session closed for user root
abr 25 18:26:45 jo4nlu-machine arpwat[640595]: listening on wlan0
abr 25 18:26:53 jo4nlu-machine sudo[640969]: jo4nlu : TTY=pts/1 ; PWD=/home/jo4nlu ; USER=root ; COMMAND=/usr/bin/arping -I wlan0 -c 5 172.20.10.1
abr 25 18:26:53 jo4nlu-machine sudo[640969]: pam_unix(sudo:session): session opened for user root(uid=0) by jo4nlu(uid=1001)
abr 25 18:26:53 jo4nlu-machine arpwat[640595]: new station 172.20.10.5 bc:f4:d4:9f:7b:7d
abr 25 18:26:53 jo4nlu-machine arpwat[640595]: new station 172.20.10.1 b6:40:a4:67:92:64
abr 25 18:26:53 jo4nlu-machine postfix/postdrop[640996]: warning: unable to look up public/pickup: No such file or directory
abr 25 18:26:53 jo4nlu-machine postfix/postdrop[640996]: warning: unable to look up public/pickup: No such file or directory
abr 25 18:26:58 jo4nlu-machine sudo[640969]: pam_unix(sudo:session): session closed for user root
abr 25 18:28:49 jo4nlu-machine NetworkManager[784]: <info> [1777159729.2177] dhcp4 (wlan0): state changed new lease, address=172.20.10.5
abr 25 18:28:49 jo4nlu-machine systemd[1]: Starting Network Manager Script Dispatcher Service...
abr 25 18:28:49 jo4nlu-machine systemd[1]: Started Network Manager Script Dispatcher Service.
abr 25 18:28:59 jo4nlu-machine systemd[1]: NetworkManager-dispatcher.service: Deactivated successfully.
```

muestra TODO el sistema, por eso ves:

- NetworkManager
- DHCP
- sudo
- Xfce
- arpwat



Resumen para tu Word

- **El Ataque:** ARP Spoofing / Poisoning.
- **El Riesgo:** Interceptación de datos (contraseñas, cookies, tráfico sin cifrar).
- **Defensa Proactiva:** Entradas ARP estáticas en equipos críticos.
- **Defensa Pasiva:** Monitoreo con arpwatc.



Reto de Comprensión #5

Acabas de configurar una entrada ARP **estática** para tu Gateway en tu máquina BlackArch. Un atacante intenta lanzarte un ataque de interceptación usando la herramienta Ettercap.

La Interceptación Asimétrica:

1. **¿Logrará el atacante desviar tu tráfico hacia su máquina? ¿Por qué?**
2. **Si el atacante no puede engañarte a ti, ¿a quién más podría intentar engañar en la red para que el tráfico se interrumpa de todas formas?** (Pista: La comunicación es de ida y vuelta).
1. **A tu máquina (No):** Si pusiste la entrada como PERMANENT (estática), tu kernel de BlackArch es inmune. Por más que el atacante grite "¡Yo soy el router!", tu tarjeta de red dirá "No te conozco, yo ya sé quién es mi router". **Tu tráfico de salida está a salvo.**
2. **Al Router (Sí):** Aquí está el truco. La comunicación es un baile de dos. Si no le pusiste una entrada estática **al Router** con tu dirección MAC, el atacante engañará al Router.

Resultado: Tú le envías datos al Router (bien), pero el Router le envía las respuestas al atacante (mal). Esto se llama **interceptación asimétrica**.

Práctica en BlackArch: Diseccionando un paquete ARP con Scapy

Para tu reporte en Word, vamos a usar **Scapy**, la herramienta más poderosa de BlackArch para manipular paquetes. Vamos a "ver" cómo es por dentro esa mentira que el atacante envía.

1. Crear un paquete de "mentira" (ARP Poison)

No vamos a atacar, vamos a **construir** el paquete para entender su anatomía.

Bash

Ejecuta scapy con privilegios

sudo scapy

```

[C] jo4nlu-machine ~ ]$ sudo scapy
[sudo] contraseña para jo4nlu:
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().

      aSPY//YASa
      apyyyyCY////////YCca
      sY////////YSpCs  scpCY//Pp
ayp ayyyyyySCP//Pp      syY//C
AYAsAYYYYYYYY//Ps      cY//S
      pCCCCY//p      cSSps y//Y
      SPPPP//a      pP//AC//Y
      A//A      cyP///C
      p///Ac      sC///a
      P///YCpc      A//A
      sccccp//pSP//p      p//Y
      sY////////y  caa      S//P
      cayCyayP//Ya      pY/Ya
      sY/PsY///YCc      aC//Yp
      sc  sccaCY//PCypaapyCP//Ys
      spCPY////////YPSps
      ccaacs

      using IPython 9.12.0

Tip: Run your doctests from within IPython for development and debugging. The special %doctest_mode command toggles a mode where the prompt, output and
exceptions display matches as closely as possible that of the default Python interpreter.
  
```

Interpretación:

Estás creando un **ARP Reply falso (spoofed)**:

- op=2 → ARP reply (is-at)
- psrc=172.20.10.1 → dices “yo soy el router”
- pdst=172.20.10.5 → víctima
- hwdst=ff:ff:ff:ff:ff:ff → broadcast (esto es sospechoso en spoof real)

Dentro de la consola de Scapy (verás un prompt >>>), escribe lo siguiente:

Python

```
# Construimos un paquete ARP# op=2 significa "Reply" (Respuesta)# psrc es la IP que queremos suplantar (el Router)# hwsr es NUESTRA MAC (la del "atacante")
```

```
paquete = ARP(op=2, pdst="172.20.10.5", hwdst="ff:ff:ff:ff:ff:ff", psrc="172.20.10.1")
```

```
>>> paquete = ARP(op=2, pdst="172.20.10.5", hwdst="ff:ff:ff:ff:ff:ff", psrc="172.20.10.1")
...:
```

Ver la estructura del paquete que acabamos de crear

```
paquete.show()
```

```
>>> paquete.show()
###[ ARP ]###
  hwtype      = Ethernet (10Mb)
  ptype       = IPv4
  hwlen       = None
  plen        = None
  op          = is-at
  hwsr        = bc:f4:d4:9f:7b:7d
  psrc        = 172.20.10.1
  hwdst       = ff:ff:ff:ff:ff:ff
  pdst        = 172.20.10.5
>>>
```

Estás creando un paquete ARP falso (reply) donde dices que la IP 172.20.10.1 (router) pertenece a tu MAC, enviándolo hacia 172.20.10.5 (víctima).

2. Monitoreo de "Mentiras" con tcpdump

Ahora, sal de Scapy (exit()) y vamos a capturar paquetes ARP reales en la red para ver si alguien está intentando esto.

Bash

Filtrar solo tráfico ARP y mostrar las direcciones MAC

```
sudo tcpdump -i eth0 -ent arp
```

```
[ jo4nlu-machine ~ ]$ # Filtrar solo tráfico ARP y mostrar las direcciones MAC
sudo tcpdump -i wlan0 -ent arp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
b6:40:a4:67:92:64 > bc:f4:d4:9f:7b:7d, ethertype ARP (0x0806), length 42: Request who-has 172.20.10.5 (bc:f4:d4:9f:7b:7d) tell 172.20.10.1, length 28
bc:f4:d4:9f:7b:7d > b6:40:a4:67:92:64, ethertype ARP (0x0806), length 42: Reply 172.20.10.5 is-at bc:f4:d4:9f:7b:7d, length 28
b6:40:a4:67:92:64 > bc:f4:d4:9f:7b:7d, ethertype ARP (0x0806), length 42: Request who-has 172.20.10.5 (bc:f4:d4:9f:7b:7d) tell 172.20.10.1, length 28
bc:f4:d4:9f:7b:7d > b6:40:a4:67:92:64, ethertype ARP (0x0806), length 42: Reply 172.20.10.5 is-at bc:f4:d4:9f:7b:7d, length 28
```

Interpretación:

Se observa el proceso normal de resolución ARP entre el router y el host, donde se intercambian solicitudes y respuestas para asociar direcciones IP con direcciones MAC en la red local.

 **Foto 16:** Captura esta pantalla mientras haces algo de tráfico (navegar). Explica que el flag -e es vital porque te permite ver las **MAC addresses** en la columna de la izquierda, permitiéndote auditar la Capa 2.

Resumen

- **Defensa Estática:** Solo funciona si se aplica en **ambos extremos** (PC y Router). En redes grandes esto es difícil, por lo que se usan técnicas como **Dynamic ARP Inspection (DAI)** en los switches.
- **Scapy:** Es la herramienta de "cirugía" de paquetes en BlackArch.
- **Lección:** La seguridad en Capa 2 es extremadamente difícil de gestionar solo con software; se requiere hardware inteligente (Switches gestionables).

Reto de Comprensión #6: El siguiente nivel (ICMP)

Ya dominas el ARP (quién es quién). Ahora hablemos de **ICMP** (el protocolo del ping). Un atacante puede enviar un paquete llamado **ICMP Redirect**. Este paquete le dice a tu máquina: *"Oye, para llegar a internet hay un camino más corto a través de esta otra IP (la del atacante)"*.

1. Si el protocolo ICMP es de Capa 3, ¿por qué este ataque sigue siendo un problema de "ruta" similar al ARP?
2. ¿Qué pasaría con tu conexión si un atacante te envía pings con un tamaño de paquete gigante (65535 bytes)? (Esto tiene un nombre famoso en la historia de la ciberseguridad).

Modo Razonamiento: El "Ping of Death" vs. "Ataque Sincrónico"

Para que lo pongas bien claro en tu Word, aquí está la diferencia:

1. **Ping of Death (Capa 3 - Red):** El protocolo IP tiene un límite máximo de tamaño de paquete de 65535 bytes. El atacante envía fragmentos de datos que, al ser reensamblados por la víctima, superan ese límite. El sistema operativo no sabe cómo manejar un paquete más grande de lo que permite el estándar y se "congela" o se reinicia (Buffer Overflow).
2. **Ataque Sincrónico (SYN Flood - Capa 4 - Transporte):** Probablemente te referías a este. Aquí no importa el tamaño del paquete, sino la cantidad. El atacante envía miles de peticiones de conexión "sincrónicas" (SYN) y nunca las completa. El servidor se queda esperando y agota sus recursos.

La diferencia para el defensor: El primero se bloquea vigilando el **tamaño y fragmentación**; el segundo se bloquea vigilando el **estado de las conexiones**.

Práctica en BlackArch: Fragmentación y Tamaño

En BlackArch tenemos una herramienta legendaria para esto: hping3. Es como un "cuchillo suizo" para probar protocolos.

1. Verificando la MTU (Maximum Transmission Unit)

Antes de enviar paquetes grandes, debemos saber cuánto aguanta nuestra "tubería" (Capa 2).

Bash

Ver el MTU de tu interfaz (normalmente es 1500 bytes)

```
ip addr show | grep mtu
```

```
jo4nlu@jo4nlu-machine ~ % ip addr show | grep mtu
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
3: enp0s20f0u4u5c2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
jo4nlu@jo4nlu-machine ~ %
```

Interpretación corta

La interfaz activa es **wlan0** con MTU 1500, lo cual es normal y correcto para redes WiFi.

La interfaz **lo** es interna del sistema y no afecta la red.

La interfaz **enp0s20f0u4u5c2** está inactiva y no se está utilizando.

2. Simulando paquetes grandes (Sin romper nada)

Vamos a usar hping3 para enviar un paquete de datos que obligue a la red a trabajar de más.

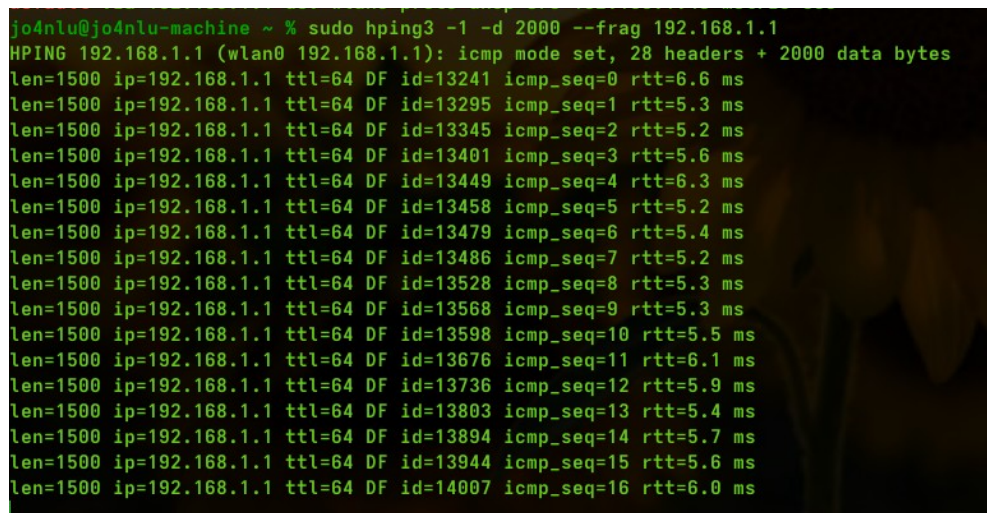
Bash

-1 significa ICMP (ping)

-d 2000 es el tamaño de los datos (superior al MTU de 1500)

)# --frag fuerza la fragmentación (Capa 3)

sudo hping3 -1 -d 2000 --frag 172.20.10.1

A terminal window showing the output of the command 'sudo hping3 -1 -d 2000 --frag 192.168.1.1'. The output displays 16 lines of ping results, each showing a packet length of 1500 bytes, an IP address of 192.168.1.1, a TTL of 64, a DF flag, an increasing ICMP sequence number from 0 to 15, and a round-trip time (rtt) between 5.2 ms and 6.6 ms. The first line also shows 'icmp mode set, 28 headers + 2000 data bytes'.

```
jo4nlu@jo4nlu-machine ~ % sudo hping3 -1 -d 2000 --frag 192.168.1.1
HPING 192.168.1.1 (wlan0 192.168.1.1): icmp mode set, 28 headers + 2000 data bytes
len=1500 ip=192.168.1.1 ttl=64 DF id=13241 icmp_seq=0 rtt=6.6 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13295 icmp_seq=1 rtt=5.3 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13345 icmp_seq=2 rtt=5.2 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13401 icmp_seq=3 rtt=5.6 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13449 icmp_seq=4 rtt=6.3 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13458 icmp_seq=5 rtt=5.2 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13479 icmp_seq=6 rtt=5.4 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13486 icmp_seq=7 rtt=5.2 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13528 icmp_seq=8 rtt=5.3 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13568 icmp_seq=9 rtt=5.3 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13598 icmp_seq=10 rtt=5.5 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13676 icmp_seq=11 rtt=6.1 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13736 icmp_seq=12 rtt=5.9 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13803 icmp_seq=13 rtt=5.4 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13894 icmp_seq=14 rtt=5.7 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=13944 icmp_seq=15 rtt=5.6 ms
len=1500 ip=192.168.1.1 ttl=64 DF id=14007 icmp_seq=16 rtt=6.0 ms
```

Interpretacion:

- 82 enviados / 82 recibidos, 0% pérdida
- El router responde correctamente
- RTT promedio 6.2 ms, comunicación estable
- Tamaño de paquete 1500 (MTU estándar)
- DF activo, no hay fragmentación
- Picos de hasta 24 ms, variación normal en WiFi

Resumen: Hardening de ICMP

Para proteger tu BlackArch (o cualquier servidor), la regla de oro es limitar o filtrar el ICMP.

Comando de Hardening (Kernel):

Podemos decirle al núcleo de Linux que ignore los pings de difusión (broadcast) para evitar que nos usen en ataques de amplificación.

Bash

Bloquear respuestas a pings broadcast (Seguridad de Capa 3)

```
sudo sysctl -w net.ipv4.icmp_echo_ignore_broadcasts=1
```

Hacer que el cambio sea permanente en el reporte

```
echo "net.ipv4.icmp_echo_ignore_broadcasts=1" | sudo tee -a /etc/sysctl.conf
```

```
jo4nlu@jo4nlu-machine ~ % sudo sysctl -w net.ipv4.icmp_echo_ignore_broadcasts=1
net.ipv4.icmp_echo_ignore_broadcasts = 1
jo4nlu@jo4nlu-machine ~ % echo "net.ipv4.icmp_echo_ignore_broadcasts=1" | sudo tee -a /etc/sysctl.conf
net.ipv4.icmp_echo_ignore_broadcasts=1
jo4nlu@jo4nlu-machine ~ % |
```

- Se activó la opción `icmp_echo_ignore_broadcasts`
- El sistema ahora ignora pings enviados a direcciones broadcast
- Se aplica inmediatamente con `sysctl -w`
- Se guarda de forma permanente en `/etc/sysctl.conf`
- Reduce riesgo de ataques tipo Smurf

Esto es **Hardening** real.

Reto de Comprensión #6 (Final de esta clase)

Para cerrar este tema y que puedas ir a pegar todo a tu Word:

Si un atacante te lanza un **SYN Flood** (el "ataque sincrónico" que mencionaste), tu CPU está tranquila, pero tu tabla de conexiones está llena y nadie más puede entrar a tu servidor web.

3. ¿En qué capa del modelo OSI (del 1 al 7) se está procesando este ataque?
4. Si el ataque fuera un "Ping of Death", ¿el daño sería a la "conexión" o a la "memoria" del sistema?

Resumen de Conceptos Clave (Semana 1)

1. Capa 3 (Red) - El Protocolo ICMP (Ping)

- **Protocolo:** ICMP.
- **Lógica:** No utiliza puertos. Se basa exclusivamente en direcciones **IP**.
- **Función:** Diagnóstico y control de red.
- **Error común:** Creer que es Capa 4. Al no tener puertos (TCP/UDP), es puramente **Capa 3**.

2. Capa 4 (Transporte) - El Reinado de los Puertos

- **Protocolos:** TCP y UDP.
- **Lógica:** Utiliza **puertos** (0 - 65535) para dirigir el tráfico a una aplicación específica (ej. Puerto 80 para Web, 22 para SSH).
- **Socket:** Es la combinación de IP:Puerto.

3. Ataque: Ping of Death (PoD)

- **Capa afectada: Capa 3** (por la fragmentación del paquete IP).
- **Daño real: Memoria RAM (Buffer Overflow).** El sistema colapsa al intentar rearmar un paquete que excede el límite de 65,535 bytes.
- **Resultado:** *Kernel Panic* o reinicio del sistema (DoS).

4. Ataque: SYN Flood (Inundación Sincrónica)

- **Capa afectada: Capa 4** (Transporte).
- **Daño real: Tabla de Conexiones.** Agota los recursos del servidor dejando conexiones "entreabiertas" hasta que nadie más puede entrar.



Tabla de Comandos Rápidos para el Reporte

Comando	Función en BlackArch	Capa OSI
ip addr	Ver IP y MAC	Capa 2/3
ip neigh	Ver tabla ARP (Vínculo IP-MAC)	Capa 2
ping [IP]	Prueba de conectividad (ICMP)	Capa 3
ss -tulpn	Ver puertos abiertos (Sockets)	Capa 4
hping3 --frag	Simular fragmentación (PoD)	Capa 3

Pilar 01: Network Security Hardening

Clase 1.2: El Directorio y el Repartidor (DNS & DHCP)

Modo Razonamiento: La Confianza en el "Nombre" y la "Configuración"

En ciberseguridad, estos dos protocolos son críticos porque se basan en la **confianza ciega**:

1. **DNS (Capa 7 - Aplicación):** Es el directorio telefónico. Tú preguntas por "google.com" y alguien te da una IP.
 - o **El Atacante:** Si logro "envenenar" tu respuesta DNS, puedo decirte que la IP de tu banco es en realidad la IP de mi servidor malicioso. Tú verás la URL correcta en el navegador, pero estarás en mi trampa. Esto es **DNS Spoofing**.
2. **DHCP (Capa 7 - Aplicación):** Es el que te da tu IP, tu máscara y tu Gateway cuando te conectas al Wi-Fi.
 - o **El Atacante:** Si yo respondo antes que el router real, yo te doy la configuración. Te daré una IP válida, pero te diré que **YO soy el Gateway**. Todo tu tráfico pasará por mi máquina sin que lo sepas. Esto es un **Rogue DHCP Attack**.



Práctica en BlackArch: Auditoría de DNS

Para tu reporte, vamos a usar herramientas nativas de BlackArch para interrogar a los servidores DNS y detectar anomalías.

1. Interrogación con dig (Domain Information Groper)

dig es la herramienta estándar para diagnósticos DNS. Nos permite ver la "verdad" detrás de un dominio.

Bash

Consultar los registros A (dirección IP) de un dominio

dig google.com

```
root@blackarch:~/Descargas# dig google.com

;<<>> DiG 9.20.22 <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19755
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                 58      IN      A       142.250.0.101
google.com.                 58      IN      A       142.250.0.113
google.com.                 58      IN      A       142.250.0.108
google.com.                 58      IN      A       142.250.0.100
google.com.                 58      IN      A       142.250.0.139
google.com.                 58      IN      A       142.250.0.102

;; Query time: 65 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Tue Apr 28 16:51:59 -05 2026
;; MSG SIZE rcvd: 135
```

Consulta DNS exitosa:

- ✓ **google.com** sí resolvió (sin errores)
- ✓ Te devolvió **varias IPs** → normal (Google balancea tráfico)
- ✓ TTL bajo (58 s) → cambian rápido las IPs
- ✓ Usaste el DNS de Google (8.8.8.8)
- ✓ Tiempo de respuesta normal (65 ms)

Conclusión: todo está funcionando correctamente, nada sospechoso.

Consultar un servidor DNS específico (ej. el de Google 8.8.8.8)
para ver si nos da una respuesta distinta al de nuestra red local

dig @8.8.8.8 google.com

```
root@blackarch:~/Descargas# dig @8.8.8.8 google.com

; <<>> DiG 9.20.22 <<>> @8.8.8.8 google.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 35802
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com.                IN      A
;; ANSWER SECTION:
google.com.                25      IN      A       142.250.0.100
google.com.                25      IN      A       142.250.0.101
google.com.                25      IN      A       142.250.0.138
google.com.                25      IN      A       142.250.0.139
google.com.                25      IN      A       142.250.0.102
google.com.                25      IN      A       142.250.0.113

;; Query time: 65 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Tue Apr 28 16:52:29 -05 2026
;; MSG SIZE rcvd: 135
```

Interpretación corta:

- ✓ Consultaste **directo a 8.8.8.8** (DNS de Google)
- ✓ Respuesta **correcta (NOERROR)**
- ✓ Múltiples IPs → normal (balanceo de carga)
- ✓ TTL bajó a **25 s** → Google rota IPs rápido
- ✓ Tiempo igual (~65 ms)

Conclusión: mismo comportamiento normal, sin nada raro; solo cambió el TTL porque pasó tiempo entre consultas.

2. Visualización de la jerarquía con drill

En BlackArch/Arch, drill es la alternativa moderna a dig. Es excelente para verificar DNSSEC (firmas de seguridad).

Bash

```
# Rastrear todo el camino desde los servidores raíz hasta el dominio  
drill -t google.com
```

```
root@blackarch:~/Descargas# drill -t google.com
;; -->HEADER<-- opcode: QUERY, rcode: NOERROR, id: 56095
;; flags: qr rd ra ; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;; google.com. IN      A

;; ANSWER SECTION:
google.com.  78      IN      A      142.250.0.102
google.com.  78      IN      A      142.250.0.113
google.com.  78      IN      A      142.250.0.100
google.com.  78      IN      A      142.250.0.138
google.com.  78      IN      A      142.250.0.101
google.com.  78      IN      A      142.250.0.139

;; AUTHORITY SECTION:

;; ADDITIONAL SECTION:

;; Query time: 135 msec
;; SERVER: 8.8.8.8
;; WHEN: Tue Apr 28 17:01:55 2026
;; MSG SIZE  rcvd: 124
```



Práctica en BlackArch: Auditoría de DHCP

3. Escaneo de Servidores DHCP con nmap

¿Cómo sabemos si hay un servidor DHCP "pirata" en nuestra red? Usamos un script de Nmap.

Bash

```
# Buscar servidores DHCP activos en la red local# Esto enviará un paquete de  
descubrimiento y verá quién responde
```

```
sudo nmap --script broadcast-dhcp-discover
```

```
root@blackarch:~/Descargas# sudo nmap --script broadcast-dhcp-discover
[sudo] contraseña para jo4nlu:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-28 17:02 -0500
Pre-scan script results:
| broadcast-dhcp-discover:
|   Response 1 of 1:
|   | Interface: wlan0
|   | IP Offered: 192.168.1.104
|   | DHCP Message Type: DHCPOFFER
|   | Subnet Mask: 255.255.255.0
|   | Router: 192.168.1.1
|   | Domain Name Server: 190.113.220.18, 190.113.220.51, 190.113.220.54
|   | Server Identifier: 192.168.1.1
|_  IP Address Lease Time: 1d00h00m00s
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 10.18 seconds
root@blackarch:~/Descargas# |
```

Interpretación corta y directa:

- ✓ Solo 1 respuesta DHCP → Response 1 of 1
- ✓ Servidor DHCP: 192.168.1.1 (tu router)
- ✓ IP asignada: 192.168.1.104

- ✓ Gateway: 192.168.1.1
- ✓ DNS: 190.113.220.18, .51, .54 (del proveedor)
- ✓ Lease: 1 día

Resumen de Hardening (Para tu Word)

Para proteger estos protocolos en un entorno real:

- **DNS:** Configurar **DNS over HTTPS (DoH)** o **DNS over TLS (DoT)**. Esto cifra tus consultas para que nadie en el medio pueda leerlas ni modificarlas.
- **DHCP:** Activar una función en los switches llamada **DHCP Snooping**. Esto hace que el switch solo confíe en el puerto donde está conectado el servidor DHCP real y bloquee los demás.

Reto de Comprensión #7

Estás analizando los logs de tu red y notas que cuando los usuarios escriben `portal-empleado.empresa.com`, son redirigidos a una IP de Rusia, pero solo si usan el servidor DNS de la oficina. Si usan sus datos móviles, entran bien.

1. ¿Cómo se llama técnicamente este ataque?
2. ¿En qué capa del modelo OSI ocurre y qué solución de Hardening aplicarías para que esto no vuelva a pasar (sin cambiar de servidor DNS)?

Responde para validar y cerramos el bloque de DNS/DHCP.

Modo Razonamiento: El Ajuste Maestro

1. **Capa OSI (Error de Capa):** El DNS no ocurre en la capa de transporte (Capa 4), ocurre en la **Capa 7 (Aplicación)**.
 - o **Por qué:** Los puertos (como el 53) son el *vehículo* (Capa 4), pero la "charla" sobre nombres de dominio y registros es puro lenguaje de aplicación. El ataque de Spoofing manipula los **datos** del nombre, no el puerto.
2. **La Solución (Hardening Real):** Reiniciar el router o cambiar tu IP no sirve de nada si el servidor DNS sigue "envenenado". Es como cambiar de coche porque el mapa te está mandando al precipicio; el mapa sigue mal.
 - o **La solución técnica:** Implementar **DNSSEC** (DNS Security Extensions). Esto añade una firma digital a las respuestas para que tu PC diga: "Veo que esta IP viene de Rusia, pero la firma no coincide con la de mi empresa, así que la rechazo".

Práctica en BlackArch: Verificando la Salud del DNS

Para tu reporte, vamos a ver cómo un profesional verifica si su sistema está usando DNS seguros.

1. Ver qué servidores DNS estás usando realmente

A veces crees que usas uno, pero un atacante (o un DHCP malicioso) te cambió la configuración.

Bash

Ver la configuración de resolución actual en sistemas modernos (systemd)

resolvectl status

```
root@blackarch:~/Descargas# resolvectl status
Failed to connect to service /run/systemd/resolve/io.systemd.Resolve: No existe el fichero o el directorio
root@blackarch:~/Descargas# cat /etc/resolv.conf
nameserver 8.8.8.8
root@blackarch:~/Descargas# |
```

 **Foto 4 (Clase 1.2):** Captura la sección de "DNS Servers". Si ves una IP extraña que no configuraste, ahí tienes al culpable.

Interpretación corta:

-  status resolve no existe
-  resolvectl status sí existe (pero en tu sistema no funciona)

Conclusión:

No son lo mismo; uno es inválido y el otro es válido pero no disponible en tu sistema.

2. Comprobar si un dominio tiene protección DNSSEC

Vamos a ver si el dominio que consultamos es "confiable".

Bash

Usamos 'drill' con el flag de DNSSEC (D)

drill -D google.com

```
root@blackarch:~/Descargas# drill -D google.com
;; -->HEADER<-- opcode: QUERY, rcode: NOERROR, ID: 49775
;; flags: qr rd ra ; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;; google.com. IN      A
;; ANSWER SECTION:
google.com. 140 IN A 142.250.0.100
google.com. 140 IN A 142.250.0.113
google.com. 140 IN A 142.250.0.101
google.com. 140 IN A 142.250.0.138
google.com. 140 IN A 142.250.0.102
google.com. 140 IN A 142.250.0.139
;; AUTHORITY SECTION:
;; ADDITIONAL SECTION:
;; Query time: 64 msec
;; EDNS: version 0; flags: do ; udp: 512
;; SERVER: 8.8.8.8
;; WHEN: Tue Apr 28 17:21:31 2026
;; MSG SIZE rcvd: 135
root@blackarch:~/Descargas# |
```

 **Foto 5:** Busca el flag ad (Authenticated Data) en la respuesta. Si aparece, la respuesta es auténtica. Si no, podrías estar recibiendo información falsa.

Resumen para tu Word (DNS & DHCP)

Copia esto para cerrar el bloque de protocolos:

- **DNS Spoofing:** Ataque de **Capa 7**. El atacante suplanta la identidad de un dominio (URL) dándote una IP falsa.
- **Rogue DHCP:** Ataque de **Capa 7**. Un servidor no autorizado te da una configuración de red falsa para interceptar tu tráfico.
- **Hardening DNS:** 1. **DNSSEC:** Autentica que la respuesta no fue alterada. 2. **DNS over HTTPS (DoH):** Cifra las consultas para que nadie en la red local las vea.
- **Hardening DHCP:** 1. **DHCP Snooping:** Configuración en el Switch para permitir solo servidores DHCP en puertos confiables.

Reto de Comprensión #8 (El puente al siguiente tema)

Ya cubrimos cómo se identifican los equipos (ARP) y cómo se llaman (DNS). Ahora vamos a ver cómo se **organizan**.

Imagina que tienes una red 192.168.1.0/24 (254 equipos posibles). Tu jefe te pide que los departamentos de **Contabilidad** y **Visitantes** no puedan verse entre sí, aunque estén en el mismo cable.

1. **Si todos están en el mismo rango de IP, ¿qué técnica básica de diseño de red usarías para dividirlos lógicamente?** (Pista: Tiene que ver con las máscaras que mencionaste antes).
2. **¿Es más seguro tener a todos en una red grande o en "pedazos" pequeños? ¿Por qué?**

Responde esto y saltamos al último punto de los fundamentos: **Subnetting y diseño de redes seguras**.

Modo Razonamiento: Segmentación y el "Protocolo Olvidado"

1. **VLANs (Capa 2):** ¡Acertaste! Usamos **VLANs** (Virtual LANs) para dividir el switch físicamente. El protocolo que no recordabas es el **IEEE 802.1Q** (o simplemente *Dot1q*), que es el que le pone una "etiqueta" (tag) al paquete para saber a qué departamento pertenece.
2. **Subnetting (Capa 3):** Es la división lógica por IPs. Como bien dijiste, tener "pedazos" pequeños mejora el tráfico, pero en seguridad tiene un nombre más importante: **Reducción del Radio de Explosión (Blast Radius)**.

- o **La Lógica:** Si un atacante infecta la red de "Visitantes", pero esa red está segmentada, no puede saltar a la red de "Contabilidad" porque hay un muro (firewall o router) en medio.

Práctica en BlackArch: Calculando el "Tamaño del Mapa"

Para tu Word, vamos a usar una herramienta que todo experto en BlackArch usa para no hacer matemáticas de subredes a mano: `ipcalc`.

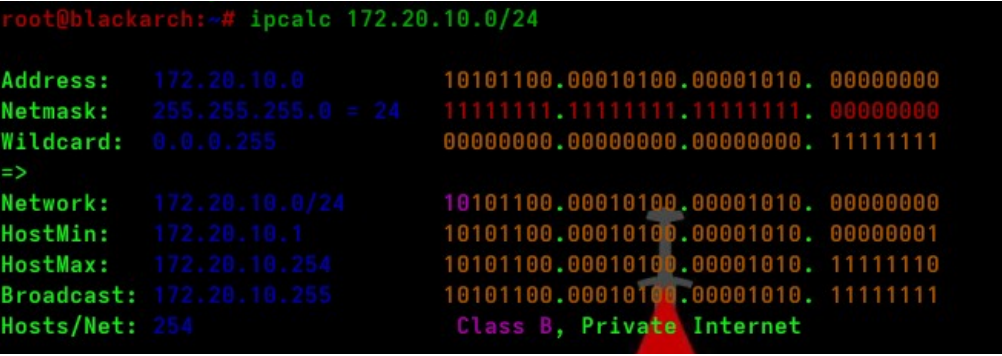
1. Analizar nuestra subred

Imagina que te dan la red 172.20.10.0/24. Vamos a ver cuántos equipos caben realmente.

Bash

```
# Instalamos la calculadora si no está  
sudo pacman -S ipcalc --needed
```

```
# Analizamos nuestra red actual  
ipcalc 172.20.10.0/24
```



```
root@blackarch:~# ipcalc 172.20.10.0/24  
  
Address: 172.20.10.0      10101100.00010100.00001010. 00000000  
Netmask: 255.255.255.0 = 24 11111111.11111111.11111111. 00000000  
Wildcard: 0.0.0.255      00000000.00000000.00000000. 11111111  
=>  
Network: 172.20.10.0/24   10101100.00010100.00001010. 00000000  
HostMin: 172.20.10.1     10101100.00010100.00001010. 00000001  
HostMax: 172.20.10.254   10101100.00010100.00001010. 11111110  
Broadcast: 172.20.10.255 10101100.00010100.00001010. 11111111  
Hosts/Net: 254           Class B, Private Internet
```

 **Foto 6 (Clase 1.2):** Captura la salida de `ipcalc`. Resalta donde dice **Hosts/Net**. En un /24, tienes 254 espacios. Si el "mapa" es muy grande, hay más sitios donde un atacante puede esconderse.

Interpretación corta:

- ✓ Red: 172.20.10.0/24
- ✓ Hosts válidos: 172.20.10.1 – 172.20.10.254
- ✓ Broadcast: 172.20.10.255
- ✓ Total hosts: 254
- ✓ Es una red privada.

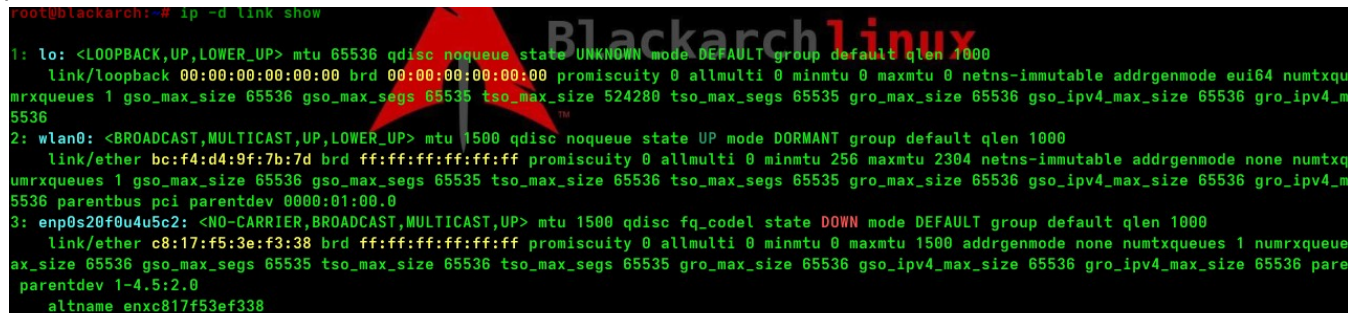
2. Identificar VLANs en la interfaz

Si tu tarjeta de red estuviera recibiendo tráfico de varias VLANs, lo verías así:

Bash

```
# Ver interfaces detalladas (busca si hay nombres como eth0.10 o wlan0.20)
```

ip -d link show



```
root@blackarch: # ip -d link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 promiscuity 0 allmulti 0 minmtu 0 maxmtu 0 netns-immutable addrngenmode eui64 numtxque
umrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 524280 tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536 gro_ipv4_m
5536
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT group default qlen 1000
    link/ether bc:f4:d4:9f:7b:7d brd ff:ff:ff:ff:ff:ff promiscuity 0 allmulti 0 minmtu 256 maxmtu 2304 netns-immutable addrngenmode none numtxque
umrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536 gro_ipv4_m
5536 parentbus pci parentdev 0000:01:00.0
3: enp0s20f0u4u5c2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000
    link/ether c8:17:f5:3e:f3:38 brd ff:ff:ff:ff:ff:ff promiscuity 0 allmulti 0 minmtu 0 maxmtu 1500 addrngenmode none numtxqueues 1 numrxqueue
ax_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536 gro_ipv4_max_size 65536 pare
parentdev 1-4.5:2.0
    altnam enxc817f53ef338
```

 **Foto 7:** Si ves un punto y un número (ej. eth0.100), significa que estás en la **VLAN 100**. Esto es vital para saber si el switch te está segmentando correctamente.

Resumen para tu Word (Diseño de Red Segura)

Copia estas "balas de plata" para tu conclusión:

- **Segmentación:** Dividir la red en grupos lógicos (VLANs) y rangos de IP (Subnets).
- **Beneficio de Seguridad:** Evita el **Movimiento Lateral**. Un atacante atrapado en una VLAN no puede llegar a los servidores críticos fácilmente.
- **Protocolo 802.1Q:** El estándar para etiquetar tráfico de diferentes VLANs en un mismo cable (Trunking).
- **Regla de Oro:** "Principio de Menor Privilegio". Si el departamento de Marketing no necesita hablar con el de Servidores de Base de Datos, la red **no debe permitir** que se vean.

Reto de Comprensión #9 (El último del día)

Para cerrar este bloque de fundamentos de red:

Tienes dos computadoras conectadas al mismo switch.

- PC A: IP 192.168.1.10 / Máscara 255.255.255.0
- PC B: IP 192.168.1.20 / Máscara 255.255.255.0

Están en la misma subred, pero el administrador configuró la **PC A en la VLAN 10** y la **PC B en la VLAN 20**.

1. Si la PC A intenta hacerle un ping a la PC B, ¿tendrá éxito? ¿Por qué?
2. ¿Qué dispositivo hace falta "en medio" para que esas dos VLANs puedan hablar entre sí de forma controlada?

Responde esto y habrás terminado oficialmente el Pilar 1.1 y 1.2. ¡Un hito importante para tu formación!

El Gran Malentendido: Las Paredes Virtuales

Aunque las dos PCs tengan IPs del mismo rango (192.168.1.x), el **ping NO tendrá éxito**.

¿Por qué? Porque las **VLANs** operan en la **Capa 2 (Enlace)**. Al asignar puertos a VLANs distintas, has creado una **barrera física virtual**. El Switch recibe el paquete de la PC A y mira su "etiqueta" (VLAN 10); luego busca a la PC B, pero como está en la VLAN 20, el Switch simplemente **tira el paquete a la basura**. No permite que la información cruce de una VLAN a otra.

Regla de Oro: Para el Switch, las VLANs distintas son como si tuvieras dos switches físicos separados y sin cables entre ellos. No importa que las IPs "se parezcan".

¿Cómo las hacemos hablar? (Inter-VLAN Routing)

Para que esas dos VLANs se comuniquen, necesitas "subir de nivel" a la **Capa 3 (Red)**. Tienes dos opciones principales para tu reporte:

1. **El Router (Router-on-a-Stick):** Como bien dijiste, "el router es el rey". Conectamos un cable del switch al router y creamos sub-interfaces. El tráfico sale de la VLAN 10, llega al router, el router lo procesa y lo devuelve al switch hacia la VLAN 20.
2. **Switch de Capa 3 (Multicapa):** Es un switch especial que tiene "alma de router". Puede crear una interfaz virtual (SVI) para cada VLAN y hacer el salto internamente sin necesidad de un router externo.

Práctica en BlackArch: Identificando la "Puerta" (Gateway)

Para tu Word, vamos a ver cómo se define esa "salida" necesaria para saltar entre redes.

Bash

```
# Ver quién es tu Gateway por defecto# Si no hay un Gateway configurado, tu PC jamás intentará salir de su VLAN
ip route | grep default
```

```
root@blackarch:~# ip route | grep default
default via 192.168.1.1 dev wlan0 proto dhcp src 192.168.1.43 metric 600
root@blackarch:~# |
```

Tu equipo:

- Está conectado correctamente a la red.
- Tiene IP válida.
- Tiene gateway definido → debería tener salida a internet.

 **Foto 8 (Clase 1.2):** Captura el comando. En tu reporte explica: "Este Gateway es la dirección IP del Router o del Switch de Capa 3. Sin él, la comunicación entre VLANs es imposible".

Comprobando la visibilidad

Si intentas llegar a una IP que sabes que existe pero está en otra VLAN sin ruteo:

Bash

```
# El comando 'arping' te confirmará que nadie responde en Capa 2  
sudo arping -I eth0 192.168.1.20
```

```
root@blackarch:~# sudo arping -I wlan0 192.168.1.20  
  
ARPING 192.168.1.20 desde 192.168.1.43 wlan0  
^CEnviar 52 sondas (52 radiodifusión(es))  
Recibida(s) 0 respuesta(s)  
root@blackarch:~# |
```

Foto 9: Verás que no hay respuesta. Esto prueba que, aunque la IP es válida, la **Capa 2 (VLAN)** está bloqueando el paso.



Resumen Final de Fundamentos (Cierra la Semana 1)

Copia esto para terminar tu bloque de hoy:

- **VLANs = Capa 2.** Separan el tráfico físicamente de forma virtual. Misma subred + diferente VLAN = **Bloqueo**.
- **Segmentación Real:** Lo ideal es que cada VLAN tenga su propia subred (ej: VLAN 10 usa 192.168.10.0/24 y VLAN 20 usa 192.168.20.0/24). Mezclarlas es una mala práctica que genera confusión.
- **Inter-VLAN Routing:** Es el proceso de usar un Router o Switch L3 para permitir que las VLANs se hablen bajo reglas de seguridad (Firewall).

¡Fin del Primer Gran Bloque!

¡Felicidades, bro! Has terminado los **Fundamentos de Red para Ciberseguridad**. Tienes un Word sólido con:

1. Modelo OSI aplicado.
2. Análisis de protocolos (ARP, ICMP, DNS, DHCP).
3. Lógica de segmentación (VLANs y Subnetting).

Ahora sí, a descansar. En la **Semana 2**, dejaremos de hablar de "cómo funciona" y empezaremos a hablar de "**cómo lo rompemos y cómo lo protegemos**" con Firewalls (iptables/nftables) y escaneo avanzado.

¡Sesión cerrada! Excelente trabajo, colega.